

Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

## **Алгоритми та методи обчислень**

### **Лабораторна робота №3**

«Інтерполяція функцій. Інтерполяційні многочлени»

Виконав: студент  
групи ІО-42  
Куліков М. М.  
Номер у списку групи: 9  
Перевірив:  
Порєв В. М.

## Лабораторна робота №3

**Тема:** «Інтерполяція функцій. Інтерполяційні многочлени».

**Мета:** Ознайомлення з інтерполяційними формулами Лагранжа, Ньютона, рекурентним співвідношенням Ейткена, методами оцінки похибки інтерполяції.

### Загальне завдання:

- Застосувати метод інтерполяції відповідно до варіанту .
- Скласти програму, що обчислює значення заданої функції  $y_i = f(x_i)$  у вузлах інтерполяції  $x_i = a + hi$ , де  $h = \frac{(b-a)}{10}$ ,  $i = 0, 1, \dots, 10$  на відрізку  $[a, b]$ .
- Передбачити в програмі оцінку похибки на основі порівняння значень, отриманих за допомогою інтерполяційних многочленів різного степеню.
- Оцінити розмитість оцінки похибки
- Налаштувати програму шляхом інтерполяції функції  $\sin x$ .
- Застосувати програму для інтерполяції функції, з таблиці 2 за номером у списку.
- Результат оцінки похибки представити у вигляді графіка і для одного з значень у вигляді таблиці.

### Індивідуальне завдання:

Завдання виконане згідно 9 варіанту.

Таблиця 1 – Завдання згідно варіанту

| Варіант | $f(x)$            | $[a, b]$ | Формула |
|---------|-------------------|----------|---------|
| 9       | $e^{-(x+\sin x)}$ | $[2, 5]$ | Ньютона |

## **Блок-схеми алгоритмів:**

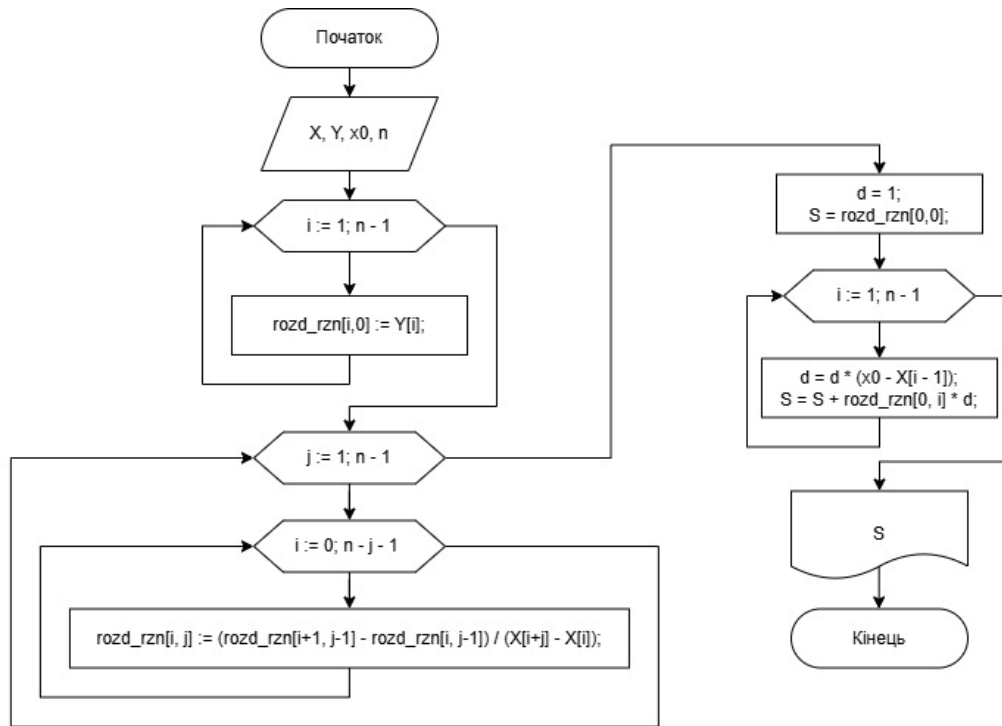


Рисунок 1 – Блок схема методу інтерполяції многочленами Ньютона

## **Код програми:**

Оскільки програма написана на WPF, і код є громіздким, то вміст усіх файлів представлено в додатку А. У цьому розділі наведений код для реалізації методу інтерполяції многочленами Ньютона.

## **Вміст файлу NewtonInterpolation.cs:**

```
internal class NewtonInterpolator {
    private readonly double[] _x;
    private readonly double[] _y;
    private readonly double[,] _d;
    private readonly int _maxDegree;
    public NewtonInterpolator(double[] x, double[] y) {
        _x = x;
        _y = y;
        _maxDegree = x.Length - 1;
        _d = new double[_maxDegree + 1, _maxDegree + 1];
        for (int i = 0; i <= _maxDegree; i++) _d[i, 0] = _y[i];
        for (int j = 1; j <= _maxDegree; j++)
            { for (int i = 0; i <= _maxDegree - j; i++) { _d[i, j] = (_d[i + 1, j - 1] - _d[i, j - 1]) / (_x[i + j] - _x[i]); } } }

    public double Evaluate(double xTarget, int degree) {
        if (degree > _maxDegree) degree = _maxDegree;
        double result = _d[0, 0];
        double term = 1.0;
        for (int i = 1; i <= degree; i++) {
            term *= (xTarget - _x[i - 1]);
            result += term * _d[0, i]; }
        return result; } }
```

## Результати виконання програми:

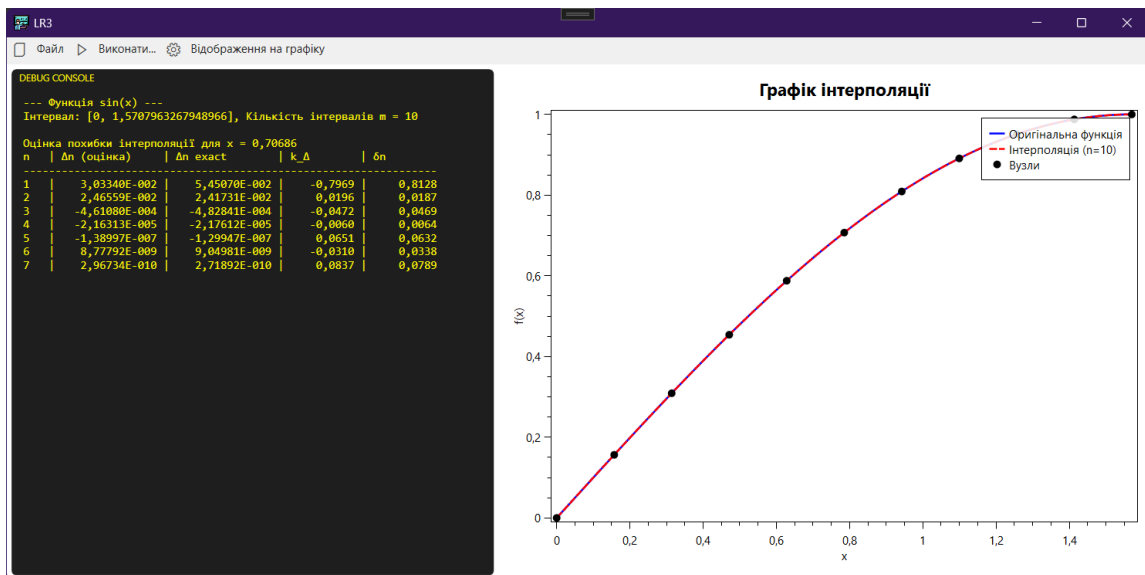


Рисунок 2 – Результат запуску інтерполяції функції  $\sin(x)$

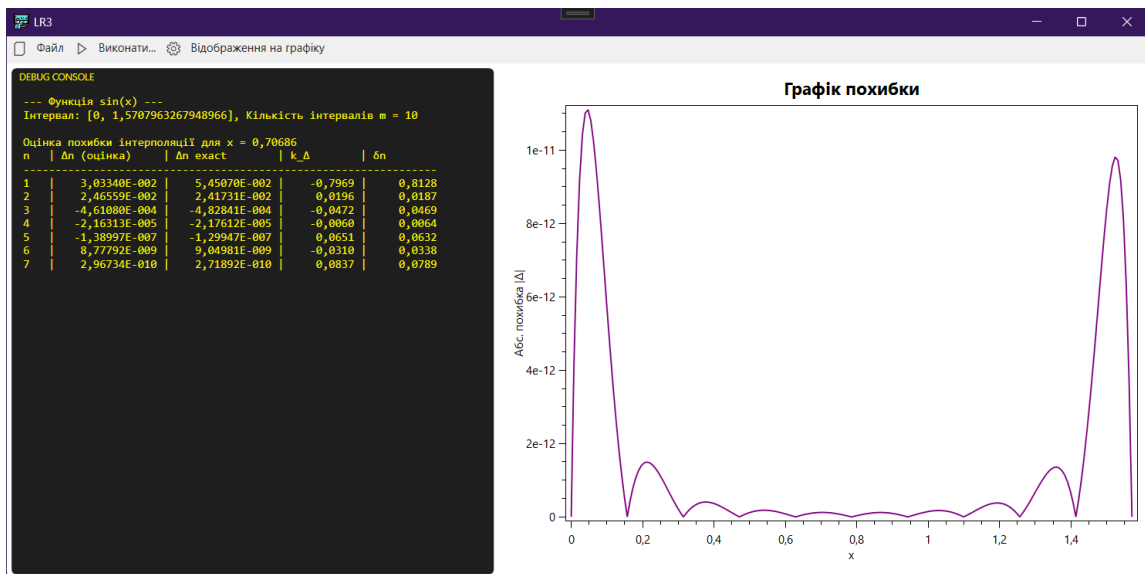


Рисунок 3 – Графік похибки при інтерполяції функції  $\sin(x)$

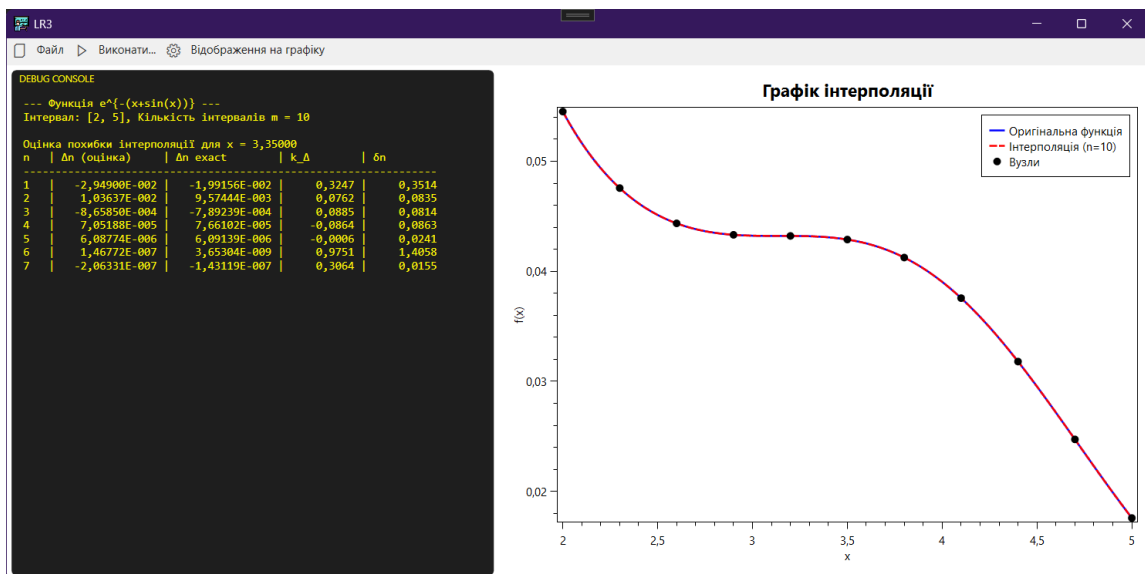


Рисунок 4 – Результат запуску інтерполяції функції  $e^{-(x+\sin x)}$

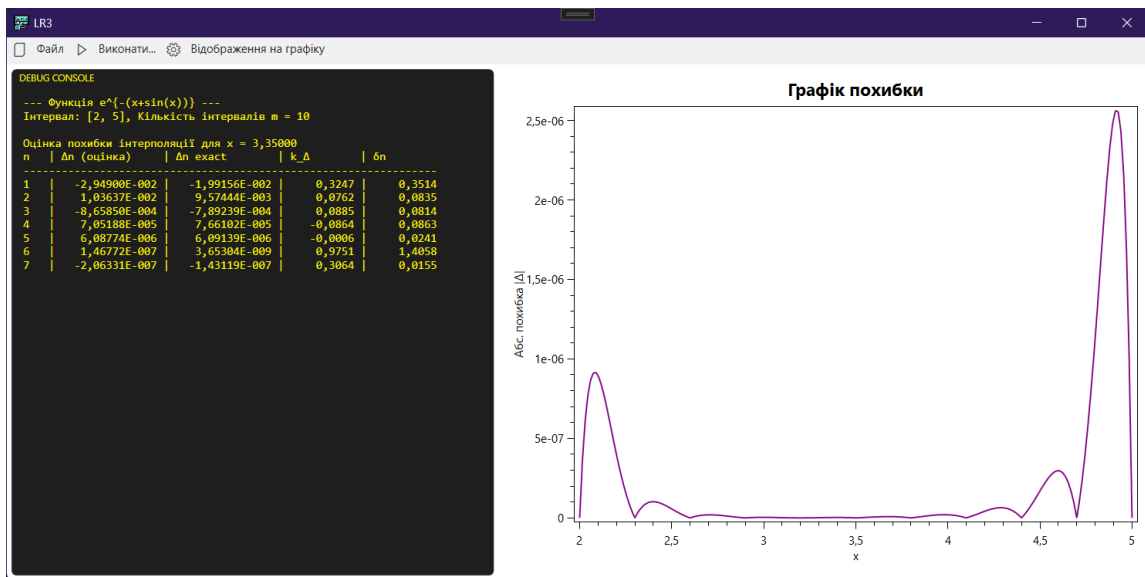


Рисунок 5 – Графік похибки при інтерполяції функції  $e^{-(x+\sin x)}$

## **Висновки:**

Отже, під час виконання лабораторної роботи було створено програму мовою C# з використанням технології WPF, яка реалізує метод інтерполяції функцій за допомогою многочленів Ньютона. Графіки оригінальних функцій та їх інтерполяційних наближень візуально збігаються на більшій частині заданих відрізків. Найбільші значення похибки спостерігаються на краях відрізка інтерполяції.

## Додаток А

### Вміст файлу MainWindow.xaml:

```
<Window x:Class="LR3.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:LR3" xmlns:oxy="clr-namespace:OxyPlot.Wpf;assembly=OxyPlot.Wpf"
    mc:Ignorable="d" Title="LR3" Height="600" Width="1200" WindowStartupLocation="CenterScreen" Icon="/icons8-
        mathematics-lineal-color-96.png">

    <DockPanel>
        <Menu DockPanel.Dock="Top" Padding="2">
            <MenuItem Header="Файл">
                <MenuItem.Icon>
                    <TextBlock FontFamily="Segoe MDL2 Assets" Text="⌂" FontSize="14" Foreground="#333333"/>
                </MenuItem.Icon>
                <MenuItem Header="Зберегти дані розрахунки" Click="SaveToFile">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="💾" FontSize="14" Foreground="#0078D7"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Вихід" Click="ExitApp">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="🚪" FontSize="14" Foreground="#E81123"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="Виконати...">
                <MenuItem.Icon>
                    <TextBlock FontFamily="Segoe MDL2 Assets" Text="⚙️" FontSize="14" Foreground="#333333"/>
                </MenuItem.Icon>
                <MenuItem Header="Тестова функція: sin(x)" Click="RunTestFunction">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="📐" FontSize="14" Foreground="#107C10"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Функція варіанту: exp(-(x+sin(x)))" Click="RunVariantFunction">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="📐" FontSize="14" Foreground="#D83B01"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Обчислити в точці x..." Click="EvaluatePoint">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="📐" FontSize="14" Foreground="#107C10"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="Відображення на графіку">
```

```

<MenuItem.Icon>
    <TextBlock FontFamily="Segoe MDL2 Assets" Text="⌂" FontSize="14" Foreground="#333333"/>
</MenuItem.Icon>
<MenuItem x:Name="mnuModeInterpolation" Header="Графік інтерполяції" IsCheckable="True" IsChecked="True"
    Click="OnGraphModeMenuClick"/>
<MenuItem x:Name="mnuModeError" Header="Графік похибки" IsCheckable="True" IsChecked="False"
    Click="OnGraphModeMenuClick"/>
</MenuItem>
</Menu>

<Grid Margin="5">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="500"/>
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <Border Grid.Column="0" Background="#1E1E1E" CornerRadius="5">
        <DockPanel>
            <TextBlock DockPanel.Dock="Top" Text="DEBUG CONSOLE" Foreground="#FFFFFF906" FontSize="10"
                Margin="5,2"/>
            <TextBox x:Name="txtLog" Background="Transparent" Foreground="#FFFFFF906"
                BorderThickness="0" VerticalScrollBarVisibility="Auto" IsReadOnly="True"
                FontFamily="Consolas" FontSize="12" Padding="10" TextWrapping="Wrap"/>
        </DockPanel>
    </Border>

    <oxy:PlotView x:Name="plotView" Grid.Column="1" Background="White" Margin="5,0,0,0"/>
</Grid>
</DockPanel>
</Window>

```

## **Вміст файлу MainWindow.xaml.cs:**

```

using LR3.Models;
using LR3.Models.Functions;
using LR3.Windows;
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;
using System.Windows;

namespace LR3
{
    public partial class MainWindow : Window
    {
        private readonly InterpolationAnalyzer _analyzer;
        private IFunctionDefinition _currentFunction;
        public MainWindow()
        {
            InitializeComponent();
            _analyzer = new InterpolationAnalyzer();
        }
    }
}

```

```

private void RunVariantFunction(object sender, RoutedEventArgs e)
{
    var inputWin = new InputWindow("Введіть кількість інтервалів (m):", "10") { Owner = this };
    if (inputWin.ShowDialog() == true && int.TryParse(inputWin.InputText, out int m) && m > 3)
    {
        _currentFunction = new CustomFunction(m);
        ExecuteAnalysis(updateLog: true);
    }
    else MessageBox.Show("Некоректне значення m. Потрібно ввести ціле число більше 3.", "Помилка введення",
        MessageBoxButton.OK, MessageBoxImage.Warning);
}

private void RunTestFunction(object sender, RoutedEventArgs e)
{
    var inputWin = new InputWindow("Введіть кількість інтервалів (m):", "20") { Owner = this };
    if (inputWin.ShowDialog() == true && int.TryParse(inputWin.InputText, out int m) && m > 3)
    {
        _currentFunction = new TestFunction(m);
        ExecuteAnalysis(updateLog: true);
    }
    else MessageBox.Show("Некоректне значення m. Потрібно ввести ціле число більше 3.", "Помилка введення",
        MessageBoxButton.OK, MessageBoxImage.Warning);
}

private void EvaluatePoint(object sender, RoutedEventArgs e)
{
    var inputWin = new InputWindow("Введіть значення x:", "3,5") { Owner = this };
    if (inputWin.ShowDialog() == true && double.TryParse(inputWin.InputText, out double x))
    {
        string result = _analyzer.EvaluateAtPoint(x);
        txtLog.AppendText(result);
        txtLog.ScrollToEnd();
    }
}

private void OnGraphModeMenuClick(object sender, RoutedEventArgs e)
{
    var clickedItem = sender as System.Windows.Controls.MenuItem;

    if (clickedItem == mnuModeInterpolation)
    {
        mnuModeInterpolation.IsChecked = true;
        mnuModeError.IsChecked = false;
    }
    else if (clickedItem == mnuModeError)
    {
        mnuModeInterpolation.IsChecked = false;
        mnuModeError.IsChecked = true;
    }
}

```

```

        if (_currentFunction != null && plotView != null)
        {
            ExecuteAnalysis(updateLog: false);
        }
    }

    private void ExecuteAnalysis(bool updateLog)
    {
        if (_currentFunction == null) return;

        bool isErrorMode = mnuModeError.IsChecked == true;
        AnalysisResult result = _analyzer.RunExperiment(_currentFunction, isErrorMode);

        plotView.Model = result.GraphModel;
        plotView.InvalidatePlot(true);

        if (updateLog)
        {
            txtLog.Clear();
            foreach (var line in result.LogLines)
            {
                txtLog.AppendText(line + "\n");
            }
            txtLog.ScrollToEnd();
        }
    }

    private void SaveToFile(object sender, RoutedEventArgs e)
    {
        var dialog = new Microsoft.Win32.SaveFileDialog { Filter = "Text Files (*.txt)|*.txt" };
        if (dialog.ShowDialog() == true)
        {
            try { DataHelper.SaveLog(dialog.FileName, txtLog.Text); MessageBox.Show("Збережено успішно!"); }
            catch (Exception ex) { MessageBox.Show(ex.Message); }
        }
    }

    private void ExitApp(object sender, RoutedEventArgs e) { Application.Current.Shutdown(); }
}

```

## **Вміст файлу InputWindow.xaml:**

```

<Window x:Class="LR3.Windows.InputWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:LR3.Windows"
    mc:Ignorable="d"

```

```

        Title="Введення" WindowStartupLocation="CenterOwner" ResizeMode="NoResize" SizeToContent="WidthAndHeight"
        MinWidth="250">
<StackPanel Margin="10">
    <TextBlock x:Name="txtPrompt" Text="Введіть значення:"/>
    <TextBox x:Name="txtInput" Margin="0,10,0,10" Text="10"/>
    <Button Content="OK" Click="Ok_Click" IsDefault="True"/>
</StackPanel>
</Window>

```

## **Вміст файлу InputWindow.xaml.cs:**

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace LR3.Windows
{
    public partial class InputWindow : Window
    {
        public string InputText { get; private set; }

        public InputWindow(string promptMessage, string defaultInput)
        {
            InitializeComponent();
            txtPrompt.Text = promptMessage;
            txtInput.Text = defaultInput;
        }

        private void Ok_Click(object sender, RoutedEventArgs e)
        {
            InputText = txtInput.Text;
            DialogResult = true;
        }
    }
}

```

## **Вміст файлу IFunction.cs:**

```

namespace LR3.Models.Functions
{
    internal interface IFunction
    {
        string Title { get; }
    }
}

```

```

double A { get; }
double B { get; }
int M { get; }
double Evaluate(double x);
}
}

```

### **Вміст файлу TestFunction.cs:**

```

namespace LR3.Models.Functions
{
    internal class TestFunction: IFunction
    {
        public TestFunction(int m) { M = m; }

        public string Title => "Функція sin(x)";
        public double A => 0;
        public double B => Math.PI / 2;
        public int M { get; private set; }
        public double Evaluate(double x) => Math.Sin(x);
    }
}

```

### **Вміст файлу CustomFunction.cs:**

```

namespace LR3.Models.Functions
{
    internal class CustomFunction: IFunction
    {
        public CustomFunction(int m) { M = m; }

        public string Title => "Функція  $e^{-(x+\sin(x))}$ ";
        public double A => 2.0;
        public double B => 5.0;
        public int M { get; private set; }
        public double Evaluate(double x) => Math.Exp(-(x + Math.Sin(x)));
    }
}

```

### **Вміст файлу AnalysisResult.cs:**

```

using OxyPlot;
namespace LR3.Models
{
    internal class AnalysisResult
    {
        public PlotModel GraphModel { get; set; }
        public List<string> LogLines { get; set; } = new List<string>();
    }
}

```

### **Вміст файлу DataHelper.cs:**

```

using System.IO;
namespace LR3.Models

```

```

{
    public static class DataHelper
    {
        public static void SaveLog(string path, string content)
        {
            if (string.IsNullOrEmpty(content))
                throw new ArgumentException("Немає даних для збереження.");

            File.WriteAllText(path, content);
        }
    }
}

```

### **Вміст файлу GraphSettings.cs:**

```

namespace LR3.Models
{
    internal class GraphSettings
    {
        public bool ShowOriginal { get; set; }
        public bool ShowInterpolation { get; set; }
        public bool ShowNodes { get; set; }
        public bool ShowError { get; set; }
    }
}

```

### **Вміст файлу InterpolationAnalyzer.cs:**

```

using LR3.Models.Functions;
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;

namespace LR3.Models
{
    internal class InterpolationAnalyzer
    {
        private NewtonInterpolator _lastInterpolator;
        private IFunction _lastFunc;

        public AnalysisResult RunExperiment(IFunction funcDef, bool isErrorMode)
        {
            _lastFunc = funcDef;
            var result = new AnalysisResult();

            result.LogLines.Add($"--- {funcDef.Title} ---");
            result.LogLines.Add($"Інтервал: [{funcDef.A}, {funcDef.B}], Кількість інтервалів m = {funcDef.M}");

            double h = (funcDef.B - funcDef.A) / funcDef.M;
            double[] X = new double[funcDef.M + 1];
            double[] Y = new double[funcDef.M + 1];

            for (int i = 0; i <= funcDef.M; i++)

```

```

{
    X[i] = funcDef.A + i * h;
    Y[i] = funcDef.Evaluate(X[i]);
}

_lastInterpolator = new NewtonInterpolator(X, Y);

var plotModel = new PlotModel
{
    Title = isErrorMode ? "Графік похибки" : "Графік інтерполяції",
    Background = OxyColors.White
};

plotModel.Axes.Add(new LinearAxis { Position = AxisPosition.Bottom, Title = "x", IsPanEnabled = false, IsZoomEnabled =
    false });

int plotPoints = 200;
double step = (funcDef.B - funcDef.A) / plotPoints;

if (isErrorMode)
{
    plotModel.Axes.Add(new LinearAxis { Position = AxisPosition.Left, Title = "Абс. похибка  $|\Delta|$ ", IsPanEnabled = false,
        IsZoomEnabled = false });

    var errorSeries = new LineSeries { Title = "Похибка", Color = OxyColors.Purple, StrokeThickness = 1.5 };

    for (double x = funcDef.A; x <= funcDef.B + 1e-9; x += step)
    {
        double exact = funcDef.Evaluate(x);
        double interp = _lastInterpolator.Evaluate(x, funcDef.M);
        errorSeries.Points.Add(new DataPoint(x, Math.Abs(exact - interp)));
    }

    plotModel.Series.Add(errorSeries);
}
else
{
    plotModel.Axes.Add(new LinearAxis { Position = AxisPosition.Left, Title = "f(x)", IsPanEnabled = false, IsZoomEnabled =
        false });

    var funcSeries = new LineSeries { Title = "Оригінальна функція", Color = OxyColors.Blue, StrokeThickness = 2 };
    var interpSeries = new LineSeries { Title = $"Інтерполяція (n={funcDef.M})", Color = OxyColors.Red, LineStyle =
        LineStyle.Dash, StrokeThickness = 2 };
    var nodesSeries = new ScatterSeries { Title = "Вузли", MarkerType = MarkerType.Circle, MarkerFill = OxyColors.Black,
        MarkerSize = 4 };

    for (int i = 0; i <= funcDef.M; i++)
    {
        nodesSeries.Points.Add(new ScatterPoint(X[i], Y[i]));
    }
}

```

```

for (double x = funcDef.A; x <= funcDef.B + 1e-9; x += step)
{
    double exact = funcDef.Evaluate(x);
    double interp = _lastInterpolator.Evaluate(x, funcDef.M);
    funcSeries.Points.Add(new DataPoint(x, exact));
    interpSeries.Points.Add(new DataPoint(x, interp));
}

plotModel.Series.Add(funcSeries);
plotModel.Series.Add(interpSeries);
plotModel.Series.Add(nodesSeries);

plotModel.Legends.Add(new OxyPlot.Legends.Legend { LegendPosition = OxyPlot.Legends.LegendPosition.TopRight,
    LegendBackground = OxyColor.FromAColor(220, OxyColors.White), LegendBorder = OxyColors.Black });
}

result.GraphModel = plotModel;

int maxDegree = funcDef.M - 3;
if (maxDegree > 0)
{
    int j_idx = funcDef.M / 2 - 1;
    double x_mid = (X[j_idx] + X[j_idx + 1]) / 2.0;

    result.LogLines.Add($"nОцінка похибки інтерполяції для x = {x_mid:F5}");
    result.LogLines.Add($"{"n",-3} | {"Δn (оцінка)",-15} | {"Δn exact",-15} | {"k_Δ",-10} | {"δn",-10}");
    result.LogLines.Add(new string('-', 65));

    for (int n = 1; n <= maxDegree; n++)
    {
        double P_n = _lastInterpolator.Evaluate(x_mid, n);
        double P_n1 = _lastInterpolator.Evaluate(x_mid, n + 1);
        double P_n2 = _lastInterpolator.Evaluate(x_mid, n + 2);

        double exact = funcDef.Evaluate(x_mid);
        double delta_n = P_n - P_n1;
        double delta_exact = P_n - exact;
        double delta_delta_n = P_n1 - P_n2;

        double k_delta = delta_n != 0 ? 1.0 - delta_exact / delta_n : 0;
        double rel_blur = delta_n != 0 ? Math.Abs(delta_delta_n / delta_n) : 0;

        result.LogLines.Add($"{"n",-3} | {delta_n,15:E5} | {delta_exact,15:E5} | {k_delta,10:F4} | {rel_blur,10:F4}");
    }
}

return result;
}

```

```

public string EvaluateAtPoint(double x)
{
    if (_lastInterpolator == null || _lastFunc == null) return "\n[!] Помилка: Спочатку виконайте аналіз функції.";
    if (x < _lastFunc.A - 1 || x > _lastFunc.B + 1) return "\n[!] Попередження: Точка x={x} знаходиться за межами відрізка
        [{_lastFunc.A}, {_lastFunc.B}]. Обчислення може бути некоректним.";
    double exact = _lastFunc.Evaluate(x);
    double interp = _lastInterpolator.Evaluate(x, _lastFunc.M);
    double error = Math.Abs(exact - interp);
    return "\n>>> Обчислення в точці x = {x}\n" + $"Оригінальна функція: {exact:F8}\n" +
        $"Поліном Ньютона: {interp:F8}\n" + $"Абсолютна похибка: {error:E5}\n";
}
}
}

```

## **Вміст файлу NewtonInterpolator.cs:**

```

namespace LR3.Models {
    internal class NewtonInterpolator {
        private readonly double[] _x;
        private readonly double[] _y;
        private readonly double[,] _d;
        private readonly int _maxDegree;

        public NewtonInterpolator(double[] x, double[] y)
        {
            _x = x;
            _y = y;
            _maxDegree = x.Length - 1;
            _d = new double[_maxDegree + 1, _maxDegree + 1];
            for (int i = 0; i <= _maxDegree; i++) _d[i, 0] = _y[i];
            for (int j = 1; j <= _maxDegree; j++)
            {
                for (int i = 0; i <= _maxDegree - j; i++) { _d[i, j] = (_d[i + 1, j - 1] - _d[i, j - 1]) / (_x[i + j] - _x[i]); } }
            }

            public double Evaluate(double xTarget, int degree)
            {
                if (degree > _maxDegree) degree = _maxDegree;
                double result = _d[0, 0];
                double term = 1.0;
                for (int i = 1; i <= degree; i++)
                {
                    term *= (xTarget - _x[i - 1]);
                    result += term * _d[0, i];
                }

                return result;
            }
        }
    }
}

```